Table of contents

---

1. combining_h5.py

loads 10x-formatted `.h5` files for individual samples and concatenates them into a single AnnData object.

files:

- **script:** `codes/combining_h5.py`
- **input files:**
  - `.h5` files (filtered feature-barcode matrix) for each sample — set `h5_path` in the script to the directory containing these files
  - expected filename pattern: `<sample_name>_filtered_feature_bc_matrix.h5` (e.g. output from Cell Ranger)
- **output files:**
  - `our_adata` — combined AnnData object (in-memory, not written to disk in final version)

process:

1. load per-sample h5 files

1. iterate over all `.h5` files in the directory matching the naming pattern `JJ0*.h5`
2. each file is read with `sc.read_10x_h5(..., gex_only=True)` to keep only gene expression features
3. `var_names_make_unique()` is called to deduplicate gene names

4. a `sample` column is added to `adata.obs` using the filename as the sample identifier

2. concatenate samples

1. the three samples (`JJ003`, `JJ004`, `JJ005`) are selected from the loaded dictionary
2. `ad.concat(..., join='outer')` merges them into a single AnnData, filling missing genes with zeros
3. `obs_names_make_unique()` ensures unique cell barcodes across samples
4. cell counts per sample are printed for verification

---

2. scanpy_pipeline.py

quality control and initial preprocessing of the combined scRNA-seq dataset, including gene-level flagging and QC metric calculation.

files:

- **script:** `codes/scanpy_pipeline.py`
- **input files:**
  - `*_filtered_feature_bc_matrix.h5` files — set `matrix_files` in the script to the directory containing these files
- **output files:**
  - `<path>/QC/violin_plot_qc.png` — violin plot of QC metrics per sample (set `path` in the script to the desired output directory)

process:

1. load samples

1. all `.h5` files ending in `_filtered_feature_bc_matrix.h5` are loaded from the matrix directory
2. each sample is annotated with a `sample` label derived from the filename
3. samples are concatenated using `ad.concat(..., join='outer')`

2. gene annotation

1. mitochondrial genes are flagged: `adata.var["mt"]` — genes starting with `MT-`
2. ribosomal genes are flagged: `adata.var["ribo"]` — genes starting with `RPS` or `RPL`
3. hemoglobin genes are flagged: `adata.var["hb"]` — genes matching `^HB[^(P)]`

3. QC metrics

1. `sc.pp.calculate_qc_metrics()` computes per-cell metrics for all three gene groups (`mt`, `ribo`, `hb`)
2. metrics include: `n_genes_by_counts`, `total_counts`, `pct_counts_mt`
3. results are computed in-place with `log1p=True`

4. visualization

1. violin plots of `n_genes_by_counts`, `total_counts`, and `pct_counts_mt` are generated, grouped by sample
2. saved to `QC/violin_plot_qc.png`

---

3. velocyto_integration.py

integrates spliced/unspliced count matrices from velocyto `.loom` files into an existing `.h5ad` AnnData object, aligning barcodes and genes across samples.

files:

- **script:** `codes/velocyto_integration.py`
- **input files:**
  - one `.loom` file per sample — output of `velocyto run` for each sample; update the `jj003_loom`, `jj004_loom`, `jj005_loom` path variables at the top of the script
  - one `.h5ad` file with existing clustering and UMAP — update the `file1` variable at the top of the script
- **output files:**
  - an `.h5ad` file with `spliced`, `unspliced`, and `ambiguous` layers added — update the output path in `adata_org.write_h5ad()`

process:

1. barcode harmonization

1. loom file cell IDs have the format `sample:barcode-x`; barcodes are extracted by splitting on `:` and stripping the trailing `x`
2. h5ad barcodes are extracted by splitting on `-` and taking the first element
3. `np.intersect1d()` finds common barcodes between each loom and the h5ad subset for that sample

2. gene harmonization

1. loom gene names are uppercased to match the h5ad (`loom_genes = [gene.upper() for gene in loom_genes]`)
2. `np.intersect1d()` finds common genes between loom and h5ad

3. genes present in h5ad but absent from the loom are tracked as `missing_genes`

3. layer extraction and padding

1. `spliced` and `unspliced` matrices are sliced from the loom using sorted gene and cell indices
2. zero-filled rows are appended for missing genes so the final matrix covers all genes in the h5ad
3. the `ambiguous` layer is initialized as all zeros (velocyto does not produce reliable ambiguous counts for this data)
4. all three matrices are transposed to cells × genes before assignment to `subset_matched.layers`

4. multi-sample alignment and integration

1. for each of the three samples, the `align_layer()` function builds a sparse zero matrix matching the full h5ad dimensions
2. shared cells and genes are identified; values are filled in at the correct indices using `get_indexer()`
3. the three aligned sparse matrices per layer are summed across samples and assigned to `adata_org.layers`

5. save output

1. the updated AnnData is written to `bc3306_with_vcy.h5ad`

---

4. subsetting_method.ipynb

subsets the scRNA-seq data by co-expression of BAT marker genes (Ebf2, Sox9, Col2a1) to identify putative brown adipocyte precursor populations, and visualizes their distribution across clusters and developmental time points.

files:

- **notebook:** `codes/subsetting_method.ipynb`
- **input files:**
  - a fully processed and clustered `.h5ad` file — update the `file` variable in cell 3 to point to your AnnData
- **output files:**
  - figures only (not written to disk explicitly)

process:

1. load and annotate clusters

1. the final AnnData is loaded and Leiden clusters (at resolution 0.4) are mapped to cell type labels
2. 23 clusters are annotated covering fibroblast progenitors, neural cells, immune cells, and mesenchymal subtypes

2. define co-expression masks

1. three co-expression groups are defined using raw counts:
   - `ebf2+_sox9+_col2a1+`: cells positive for all three markers (Ebf2, Sox9, Col2a1)
   - `ebf2+_sox9+_col2a1-`: cells positive for Ebf2 and Sox9 but negative for Col2a1
2. boolean masks are derived from `adata.raw[:, gene].X.todense() > 0`
3. a combined categorical column `ebf2_sox9_col2a1` is added to `adata.obs` for joint UMAP visualization

3. spatial distribution across clusters

1. the count of `ebf2+/sox9+/col2a1-` cells per Leiden cluster is tabulated and plotted as a bar chart
2. fibroblast progenitor clusters (0, 1, 6) are isolated into `FP_adata_subset`
3. Pdgfra+ clusters (0, 1, 3, 6, 13, 14) are isolated into `pdgfra_adata_subset`

4. BAT marker expression visualization

1. expression of BAT markers (`Ebf2`, `Sox9`, `Cdh4`, `Pparg`, `Hoxa5`, `Gata6`, `Cebpa`) is plotted on UMAP for each subset
2. each subset is further filtered to the `ebf2+_sox9+_col2a1-` condition and markers are re-visualized

5. temporal labeling

1. samples are relabeled by embryonic day: `JJ005` → `E11.5`, `JJ004` → `E12.5`, `JJ003` → `E13.5`
2. for each BAT marker, a per-cell column `gene_day` is created that reports the sample's day for expressing cells and `None` for non-expressing cells
3. these columns are plotted on UMAP with a custom 3-color palette by developmental stage

---

5. scvelo_analysis.ipynb

RNA velocity analysis of Pdgfra+ mesenchymal clusters using the scVelo dynamical model, including velocity embedding, latent time, kinetic rate estimation, and identification of dynamical genes.

files:

- **notebook:** `codes/scvelo_analysis.ipynb`
- **input files:**
  - an `.h5ad` file with `spliced` and `unspliced` layers — output of `velocyto_integration.py`; update the `file` variable in cell 3
- **output files:**
  - an `.h5ad` file with all velocity results — update the output path in the final `write_h5ad()` call
  - a `.csv` file with top dynamical genes for the cluster of interest — update the output path in the `to_csv()` call

process:

1. subset to Pdgfra+ clusters

1. clusters `0`, `1`, `3`, `6`, `13`, `14` (Pdgfra+) are selected from the full dataset
2. re-clustering at resolution 0.6 produces 13 sub-clusters
3. sub-clusters are manually annotated to cell types: Brown adipocytes, Fibroblast progenitors, Tendon, Muscle connective tissue, Chondrocytes, Dermis, Meninges, Smooth muscle

2. scVelo preprocessing

1. `scv.pp.filter_and_normalize()` filters genes and normalizes counts
2. `scv.pp.moments()` computes first- and second-order moments using `n_pcs=30`, `n_neighbors=30`

3. dynamical model fitting

1. `scv.tl.recover_dynamics()` fits the full transcriptional dynamics model per gene using `n_jobs=4`
2. the model estimates gene-specific kinetic parameters: transcription rate (`fit_alpha`), splicing rate (`fit_beta`), degradation rate (`fit_gamma`), switching time, and scaling

4. velocity computation

1. `scv.tl.velocity(..., mode="dynamical")` computes RNA velocities from the fitted model
2. `scv.tl.velocity_graph()` builds a cell-cell transition graph based on velocity cosine similarities
3. differential kinetics (`diff_kinetics=True`) are applied in a second pass to account for cluster-specific dynamics

5. latent time and pseudotime

1. `scv.tl.velocity_pseudotime()` assigns pseudotime based on the velocity graph
2. `scv.tl.latent_time()` computes a universal gene-shared latent time approximating the real internal cell clock

6. dynamical gene ranking

1. `scv.tl.rank_dynamical_genes()` ranks genes by fit likelihood per cell type
2. top dynamical genes for the Brown adipocyte cluster are exported to CSV
3. a heatmap of these genes sorted by latent time is generated using `scv.pl.heatmap()`

---

6. rna_velo_analysis.ipynb

extended RNA velocity analysis building on the scvelo_analysis results, with additional phase portrait analysis, velocity confidence metrics, PAGA trajectory inference, and kinetic rate visualization.

files:

- **notebook:** `codes/rna_velo_analysis.ipynb`
- **input files:**
  - an `.h5ad` file with `spliced` and `unspliced` layers — output of `velocyto_integration.py`; update the `file` variable in cell 2
- **output files:**
  - an `.h5ad` file with velocity and latent time results — update the output path in the `write_h5ad()` call
  - a `.csv` file with top 300 dynamical genes for the cluster of interest — update the output path in the `to_csv()` call

process:

1. subset and annotate

1. Pdgfra+ clusters are subsetted identically to `scvelo_analysis.ipynb`
2. re-clustering at resolution 0.6 and manual annotation produces 13 named cell types

2. velocity computation

1. scVelo preprocessing, moment computation, dynamics recovery, and velocity embedding follow the same steps as `scvelo_analysis.ipynb`
2. velocity embedding stream plots are generated on UMAP, colored by cell type

3. phase portraits

1. `scv.pl.velocity()` generates phase portraits (spliced vs. unspliced) for marker genes including `PPARG`, `SOX9`, `EBF2`, `HOXA5`, `CEBPA`, `CDH4`, `GATA6`
2. `scv.pl.scatter()` overlays velocity values and cell types on the phase portrait

4. velocity gene ranking

1. `scv.tl.rank_velocity_genes()` identifies genes that drive velocity differences between clusters (`min_corr=0.3`)
2. top 5 BA-associated velocity genes are visualized with phase portraits

5. velocity confidence and speed

1. `scv.tl.velocity_confidence()` computes per-cell velocity length (speed) and confidence (coherence with neighbors)
2. per-cluster means are displayed as a styled table

6. cell transitions and PAGA

1. `scv.utils.get_cell_transitions()` traces stochastic cell transitions starting from a specified cell
2. PAGA with velocity-informed edge weights is computed via `scv.tl.paga()` for both leiden sub-clusters and cell type annotations
3. PAGA graphs are overlaid on UMAP

7. kinetic rate distributions

1. transcription, splicing, and degradation rate histograms are plotted for genes with `fit_likelihood > 0.1`
2. `scv.tl.latent_time()` computes latent time; top dynamical genes for BA are ranked and exported

8. gene heatmaps

1. heatmaps of the top 300 BA dynamical genes are generated sorted by latent time using both spliced (`Ms`) and unspliced (`Mu`) moment layers

---

7. tangram_test_visium.ipynb

maps scRNA-seq cell types onto Visium spatial transcriptomics spots using Tangram, then projects single-cell gene expression into spatial coordinates and evaluates mapping quality.

files:

- **notebook:** `codes/tangram_test_visium.ipynb`
- **input files:**
  - a reference scRNA-seq `.h5ad` with Leiden cluster annotations (`adata_sc`) — update the path in cell 2
  - a Visium spatial `.h5ad` with spatial coordinates and cluster annotations (`adata_st`) — update the path in cell 2
- **output files:**
  - an `.h5ad` file storing the cell-to-spot mapping probability matrix — update the output path in the `write_h5ad()` call for `ad_map`
  - an `.h5ad` file with gene expression projected from single cells to Visium spots — update the output path in the `write_h5ad()` call for `ad_ge`

process:

1. load data

1. the scRNA-seq reference and Visium target datasets are loaded separately
2. spatial clustering results from Visium (`leiden_expr_spatial_scaled`) and scRNA-seq UMAP (`leiden_0.4`) are visualized side by side

2. select training genes

1. marker genes are identified using `sc.tl.rank_genes_groups()` on `leiden_0.4` clusters
2. the top 100 markers per cluster are collected and deduplicated, yielding the training gene set

3. Tangram preprocessing

1. `tg.pp_adatas()` filters both AnnData objects to the shared training gene set and computes necessary metadata

4. cell-to-space mapping

1. `tg.map_cells_to_space()` is run in `mode="cells"` with `density_prior="rna_count_based"` and `num_epochs=500`
2. device is set to MPS (Apple Silicon GPU) if available, else CPU
3. the resulting `ad_map` stores a cells × spots probability matrix

5. evaluate mapping quality

1. `tg.plot_training_scores()` produces four diagnostic panels: score histogram, score vs. scRNA-seq sparsity, score vs. spatial sparsity, and score vs. sparsity difference

2. `tg.project_cell_annotations()` projects cluster labels from scRNA-seq onto spots
3. `tg.plot_cell_annotation_sc()` visualizes per-spot cell type composition

6. gene expression projection

1. `tg.project_genes()` uses the mapping matrix to project single-cell gene expression onto Visium spots, producing `ad_ge`
2. `tg.plot_genes_sc()` compares predicted vs. measured expression for selected genes (`sox9`, `ebf2`, `cxcl12`, `hoxa5`)

7. AUC evaluation

1. `tg.compare_spatial_geneexp()` scores all genes by comparing Tangram-predicted expression to measured Visium expression
2. `tg.plot_auc()` visualizes the resulting AUC curve

8. deconvolution (exploratory)

1. the H&E image from the Visium dataset is smoothed and segmented using a watershed algorithm via `squidpy`
2. results are visualized alongside DAPI and cluster overlays

---

8. tangram_test_cosmx.ipynb

maps scRNA-seq cell types onto CosMx spatial transcriptomics data using Tangram, then projects gene expression and evaluates predicted vs. measured expression across the full tissue.

files:

- **notebook:** `codes/tangram_test_cosmx.ipynb`
- **input files:**
  - a reference scRNA-seq `.h5ad` with Leiden cluster annotations (`adata_sc`) — update the path in cell 2
  - a CosMx spatial `.h5ad` with `CenterX_global_px` and `CenterY_global_px` columns in `.obs` (`adata_st`) — update the path in cell 2
- **output files:**
  - an `.h5ad` file storing the cell-to-CosMx-cell mapping probability matrix — update the output path in the `write_h5ad()` call for `ad_map`

parameters:

parameter	value
`n_obs` (downsampling)	30,000
`num_epochs`	500
`density_prior`	`rna_count_based`
`mode`	`cells`
`device`	`cpu`
top markers per cluster	100

process:

1. load data

1. scRNA-seq reference and CosMx spatial datasets are loaded
2. CosMx is downsampled to 30,000 cells using `sc.pp.subsample()` to reduce computational cost

2. select training genes

1. `sc.tl.rank_genes_groups()` identifies marker genes per Leiden cluster (`leiden_0.4`) using normalized expression
2. top 100 markers per cluster are collected and deduplicated

3. Tangram preprocessing and mapping

1. `tg.pp_adatas()` aligns both datasets to the shared training gene set
2. `tg.map_cells_to_space()` maps scRNA-seq cells to CosMx cells in `mode="cells"` with 500 epochs on CPU

4. gene expression projection

1. `tg.project_genes()` projects single-cell gene expression onto CosMx spatial coordinates using `ad_map`
2. spatial coordinates from CosMx (`CenterX_global_px`, `CenterY_global_px`) are transferred to `ad_ge.obsm["spatial"]`

5. spatial visualization

1. per-FOV plots compare measured vs. Tangram-predicted expression using `sq.pl.spatial_segment()` for a single FOV
2. stitched full-tissue scatter plots overlay SOX9 expression on DAPI background for both measured and predicted data, using a shared color scale for direct comparison

6. AUC evaluation

1. `tg.compare_spatial_geneexp()` scores all genes by comparing Tangram predictions to CosMx measurements

2. `tg.plot_auc()` generates the AUC curve for the full gene panel

---

9. combined_modalities_analysis.ipynb

CellRank-based cell fate analysis combining RNA velocity (VelocityKernel) and optimal transport over real developmental time (RealTimeKernel via moscot), identifying terminal states, fate probabilities, and driver genes for the Brown adipocyte lineage.

files:

- **notebook:** `codes/combined_modalities_analysis.ipynb`
- **input files:**
  - an `.h5ad` file with scVelo velocity layers and latent time — output of `scvelo_analysis.ipynb` or `rna_velo_analysis.ipynb`; update the `file` variable in cell 2
- **output files:**
  - a `.csv` file with CellRank lineage driver genes for the terminal state of interest — update the output path in the `to_csv()` call

process:

1. load data and time annotation

1. the scVelo-analyzed AnnData is loaded; sample labels are mapped to embryonic days (`JJ005` → `11.5`, `JJ004` → `12.5`, `JJ003` → `13.5`)
2. `day_numerical` is added as a float column for use by moscot's temporal solver

2. RealTimeKernel — optimal transport over developmental time

1. `TemporalProblem` is initialized and `score_genes_for_marginals()` scores each cell for proliferation and apoptosis using mouse gene sets — these scores serve as growth rates for the optimal transport problem
2. `tp.prepare(time_key="day_numerical")` sets up the inter-timepoint transport problems (E11.5→E12.5, E12.5→E13.5)
3. `tp.solve(epsilon=1e-3, tau_a=0.95, scale_cost="mean")` solves the entropic regularized optimal transport; `tau_a=0.95` allows mild unbalancedness
4. `RealTimeKernel.from_moscot(tp)` converts the transport plan to a CellRank transition matrix
5. `compute_transition_matrix()` is called with `conn_weight=0.2` to blend with a KNN connectivity graph

3. VelocityKernel — RNA velocity transitions

1. `cr.kernels.VelocityKernel` is initialized from the scVelo velocity graph
2. `compute_transition_matrix()` produces a velocity-based cell-cell transition matrix

4. combined kernel

1. the two kernels are linearly combined with equal weights: `0.5 * vk + 0.5 * tmk`
2. the combined kernel captures both RNA velocity dynamics and temporal fate transitions

5. macrostate and terminal state identification

1. the GPCCA estimator is applied to each kernel independently and to the combined kernel
2. `g.fit()` / `c.fit()` identifies macrostates (`n_states=[8, 10]`); terminal states are predicted automatically
3. for the combined kernel, terminal states are set manually to ensure biological interpretability: Brown adipocytes, Muscle connective tissue, Dermis ($\times 2$), Smooth muscle, Chondrocytes, Meninges, Tendon

6. fate probabilities

1. `compute_fate_probabilities()` assigns each cell a probability of reaching each terminal state
2. BA fate probabilities are added to `adata_velo.obs` and visualized on UMAP

7. lineage driver gene identification

1. `compute_lineage_drivers()` correlates gene expression with BA fate probabilities across relevant clusters
2. driver genes are ranked by correlation; the top 80 are visualized in a heatmap sorted by scVelo latent time using a GAMR smoothing model
3. results are exported to CSV